LAI LONG CHARTS — Tài liệu hệ thống

Cấu trúc · Logic · Vận hành · Bài tập

© 2026 Lai Long Charts. Bảo lưu mọi quyền.

Trang web: https://lehoanglong2452004-eng.github.io/lai-long-charts/

Mã nguồn: https://github.com/lehoanglong2452004-eng/lai-long-charts

---

Mục lục

1. Tổng quan: hệ thống làm gì
2. Kiến trúc: các mảnh ghép và cách chúng nói chuyện với nhau
3. Cấu trúc thư mục: file nào làm gì
4. Phần dữ liệu (backend): GitHub Actions và các script Python
5. Phần giao diện (frontend): các module JavaScript
6. Logic tab COT
7. Logic tab Mùa vụ (Seasonal)
8. Logic tab Biểu đồ (Charts)
9. Dữ liệu realtime đa sàn và cơ chế tự chuyển sàn
10. Vận hành hằng ngày và xử lý sự cố
11. Hướng dẫn mở rộng: thêm sản phẩm, thêm sàn, đổi lịch
12. Bài tập tự học
13. Phụ lục: bảng mã, định dạng file, từ điển thuật ngữ

---

1. Tổng quan

Lai Long Charts là một "terminal" theo dõi thị trường hàng hóa, tiền tệ, năng lượng và crypto, với phong cách giống Bloomberg. Trang có 3 tab chính:

Tab	Mục đích	Dữ liệu dùng
COT	Xem vị thế của các nhóm nhà đầu tư (đầu cơ, thương mại, nhỏ lẻ...) trên thị trường futures Mỹ	Báo cáo Commitments of Traders của CFTC, ra mỗi tuần
Mùa vụ	So sánh đường giá năm nay với các năm trước, so sánh nhiều sản phẩm theo %	Giá đóng cửa hằng ngày từ năm 2000
Biểu đồ	Đa biểu đồ nến/đường, volume, volume profile, đồng hồ đếm ngược	Realtime từ 4 sàn crypto, và giá trễ ~15 phút từ Yahoo

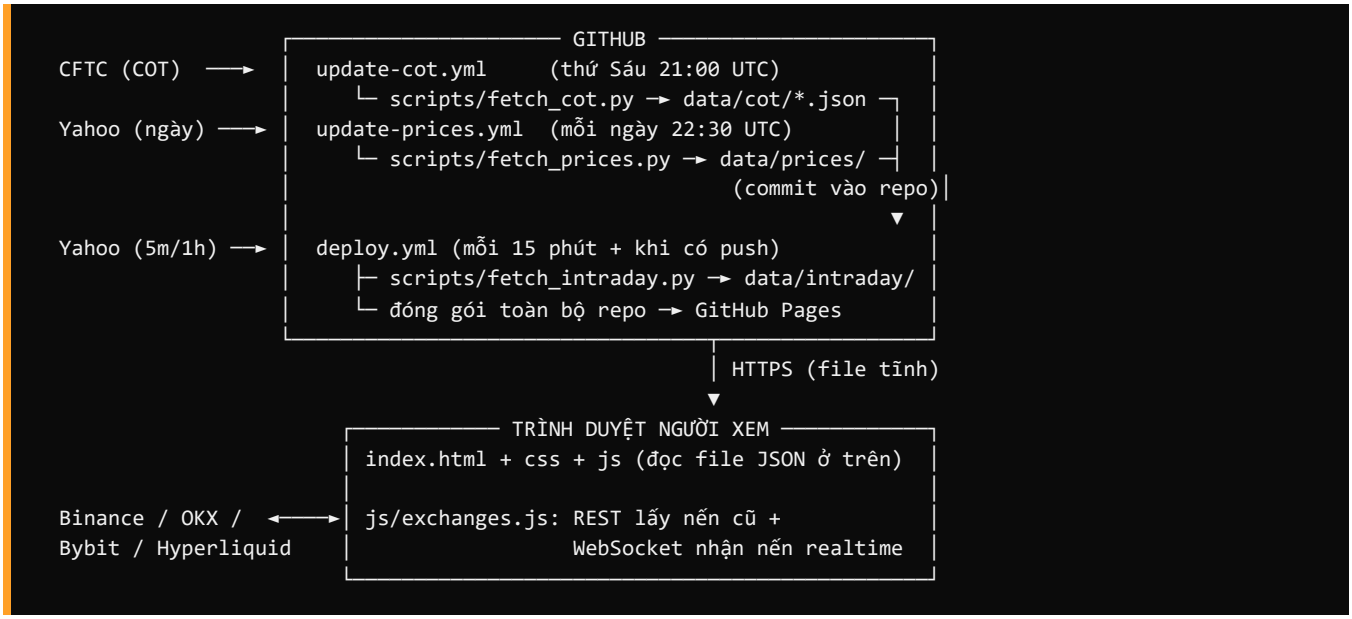
Điểm đặc biệt của hệ thống là **không có máy chủ (server)**. Toàn bộ chạy trên hạ tầng miễn phí của GitHub:

- **GitHub Actions** đóng vai trò "robot" đi lấy dữ liệu theo lịch.
- **GitHub Pages** đóng vai trò "web hosting", phát trang web và file dữ liệu.
- **Trình duyệt của người xem** tự kết nối thẳng tới các sàn để lấy giá realtime.

---

2. Kiến trúc

2.1 Sơ đồ tổng thể



2.2 Ba loại dữ liệu, ba cách vận chuyển

Loại	Tần suất	Cách lưu	Vì sao
COT, giá ngày	Tuần / ngày	**Commit vào repo** (thư mục `data/`)	Ít thay đổi, cần giữ lịch sử, xem lại được trên GitHub
Giá trong ngày 5m / 1h (futures, forex)	15 phút	**Không commit**, chỉ đóng gói khi deploy	Thay đổi liên tục; commit 96 lần/ngày sẽ làm repo phình to
Realtime (crypto, kim loại, năng lượng perp)	Từng giây	**Không lưu**, trình duyệt nhận trực tiếp	Không cần máy chủ trung gian

2.3 Chuỗi tự động (quan trọng)

1. `update-prices.yml` chạy, commit file giá mới vào repo.
2. Ở bước cuối, nó gọi `gh workflow run deploy.yml`.
3. `deploy.yml` lấy dữ liệu intraday mới, đóng gói rồi xuất bản lên Pages.
4. Ngoài ra, `deploy.yml` tự chạy **mỗi 15 phút**, nên intraday luôn được làm mới.

> **Vì sao bước 2 cần thiết?** Commit do bot (`GITHUB_TOKEN`) tạo ra **không** tự kích hoạt workflow khác (GitHub chặn để tránh vòng lặp vô hạn). Riêng lệnh `workflow run` thì được phép, nên ta gọi deploy một cách tường minh.

---

3. Cấu trúc thư mục

```
lai-long-charts/
├── index.html                Khung trang: thanh trên, ticker tape, vùng nội dung, chân trang, hộp cài
đặt
├── css/style.css             Toàn bộ giao diện: màu theo theme, bố cục, responsive
├── js/
│   ├── app.js                Điểm khởi động: chuyển tab, theme, ngôn ngữ, ticker tape, đồng hồ, cài đặt
│   ├── config.js             Danh mục sản phẩm, nhóm COT, bảng màu
│   ├── i18n.js               Từ điển EN/VI
│   ├── util.js               Hàm dùng chung: tải JSON, định dạng số, tạo chart, localStorage
│   ├── ui.js                 Thành phần giao diện: nút phân đoạn, sidebar sản phẩm, nút toàn màn hình
│   ├── cot.js                Tab COT
│   ├── seasonal.js           Tab Mùa vụ
│   ├── charts.js             Tab Biểu đồ
│   ├── exchanges.js          Kết nối 4 sàn + cơ chế tự chuyển sàn
│   └── vendor/lightweight-charts.js   Thư viện vẽ chart của TradingView (v5.0.9)
├── scripts/
│   ├── fetch_cot.py          Tải báo cáo COT từ CFTC
│   ├── fetch_prices.py       Tải giá ngày từ Yahoo
│   ├── fetch_intraday.py     Tải giá 5m / 1h từ Yahoo
│   └── requirements.txt      Thư viện Python: requests, pandas, yfinance
├── data/
│   ├── cot/{legacy,disagg,tff}/*.json   Lịch sử COT từng thị trường
│   ├── cot/index.json                   Thị trường nào có loại báo cáo nào
│   ├── prices/*.json                    Giá ngày từng sản phẩm
│   ├── prices/index.json                Tóm tắt giá cuối, dùng cho ticker tape
│   └── intraday/  (không có trong repo, chỉ có trên trang đã deploy)
├── .github/workflows/
│   ├── update-cot.yml        Lịch COT
│   ├── update-prices.yml     Lịch giá ngày
│   └── deploy.yml            Xuất bản trang + intraday
├── docs/                     Tài liệu này
├── .gitignore                Bỏ qua data/intraday/
├── .nojekyll                 Báo GitHub Pages không xử lý Jekyll
├── LICENSE, README.md
```

---

4. Phần dữ liệu (backend)

4.1 `fetch_cot.py`: báo cáo COT

Nguồn: file năm của CFTC tại `https://www.cftc.gov/files/dea/history/`. Mỗi năm có 3 file zip:

Báo cáo	Tên file	Nhóm trader
Legacy	`deacot{năm}.zip`	Non-Commercial (đầu cơ), Commercial (thương mại), Non-Reportable (nhỏ lẻ)
Disaggregated	`fut_disagg_txt_{năm}.zip`	Producer/Merchant, Swap Dealer, Managed Money, Other Reportables
TFF	`fut_fin_txt_{năm}.zip`	Dealer, Asset Manager, Leveraged Funds, Other

> **Vì sao không dùng API của CFTC?** API `publicreporting.cftc.gov` trả lỗi 403 với IP ngoài Mỹ. File zip thì tải được ở mọi nơi, nên hệ thống chạy được cả khi thử trên máy ở Việt Nam.

Logic:

1. Với mỗi loại báo cáo, tải file từ năm 2010 đến năm hiện tại.
2. Đọc CSV và lọc dòng theo **mã hợp đồng CFTC** (ví dụ vàng = `088691`).
3. Chỉ giữ các cột vị thế cần thiết, đổi tên ngắn (ví dụ `nc_l` = Non-Commercial Long).
4. Ghi ra JSON theo dạng **cột** (columnar): `{"date":[...], "oi":[...], "nc_l":[...], ...}`.
5. Ghi `data/cot/index.json` cho biết thị trường nào có báo cáo nào.
6. Nếu một thị trường không có dữ liệu thì in `MISSING` và **thoát với mã lỗi**, để GitHub báo email cho bạn.

> **Vì sao lưu dạng cột?** File nhỏ hơn rất nhiều so với mảng object (tên trường chỉ xuất hiện một lần), và JavaScript đọc từng cột để vẽ chart rất tiện.

4.2 `fetch_prices.py`: giá ngày

- Dùng thư viện `yfinance` gọi `Ticker(tk).history(start="2000-01-01", interval="1d")`.
- Mỗi sản phẩm ghi một file `data/prices/{SYMBOL}.json` với các cột `t,o,h,l,c,v`.
- Tạo `data/prices/index.json` gồm `close` (giá cuối) và `prev` (giá phiên trước) cho ticker tape.
- Có thêm các hợp đồng tiền tệ CME (`6E=F`, `6J=F` ...) để vẽ trên tab COT. Lý do: COT của Yên là hợp đồng JPY/USD, ngược chiều với cặp USDJPY quen thuộc.
- Chỉ báo lỗi khi **hơn một nửa** sản phẩm thất bại, vì khi đó nhiều khả năng Yahoo đang sập. Một ticker hỏng thì không làm hỏng cả lần chạy.

4.3 `fetch_intraday.py`: giá 5 phút và 1 giờ

- Với 5m lấy 15 ngày gần nhất, với 1h lấy 1 năm.
- Bỏ qua BTC/ETH, vì trình duyệt lấy realtime trực tiếp từ sàn.

- **Phát hiện lỗ hổng dữ liệu:** Yahoo đôi khi trả thiếu nguyên một phiên. Hàm `recent_gap()` kiểm tra 48 giờ gần nhất xem có khoảng trống trên 6 giờ không rơi vào cuối tuần. Nếu có thì tải lại, tối đa 3 lần, và giữ bản nhiều dữ liệu nhất.
- Thời gian lưu bằng **giây Unix (UTC)**.

4.4 Ba workflow

File	Kích hoạt	Việc làm
`update-cot.yml`	cron `0 21 * * 5` (thứ Sáu) và `0 3 * * 6` (dự phòng), hoặc bấm tay	Chạy `fetch_cot.py`, commit nếu có thay đổi, gọi deploy
`update-prices.yml`	cron `30 22 * * *` (mỗi ngày), hoặc bấm tay	Chạy `fetch_prices.py`, commit, gọi deploy
`deploy.yml`	push lên `main`, cron `*/15 * * * *`, bấm tay	Chạy `fetch_intraday.py`, upload artifact, deploy Pages

Các chi tiết cần hiểu:

- `permissions: contents: write` cho phép bot commit. `actions: write` cho phép bot gọi workflow khác.
- `concurrency: data-update` để hai workflow dữ liệu không ghi đè nhau.
- Dòng `git diff --cached --quiet || (git commit ... && git pull --rebase && git push)` nghĩa là chỉ commit khi có thay đổi, và kéo bản mới trước khi đẩy để tránh xung đột.
- `continue-on-error: true` ở bước intraday: nếu Yahoo lỗi thì trang vẫn được deploy, chỉ thiếu intraday mới.
- Cron trên GitHub dùng **giờ UTC** (giờ Việt Nam = UTC+7) và có thể trễ 5–30 phút khi hệ thống đông.

---

5. Phần giao diện (frontend)

5.1 Cách trang khởi động

1. `index.html` nạp `lightweight-charts.js` (script thường, tạo biến toàn cục `LightweightCharts`).
2. Sau đó nạp `app.js` dạng **ES module** (`type="module"`), và nó `import` các module khác.
3. `app.js` gọi lần lượt: `applyTheme()`, `applyLang()`, `initSettings()`, `buildTape()`, `route()`, `tick()`.

5.2 Định tuyến tab (`route()` trong app.js)

- Tab hiện tại nằm trên URL sau dấu `#`: `#cot`, `#seasonal`, `#charts`.
- Mỗi tab là một module có hàm `mount(container)`, và hàm này **trả về một hàm dọn dẹp** (`destroy`).
- Khi đổi tab: gọi `destroy()` của tab cũ (đóng WebSocket, xóa chart, dừng timer), xóa nội dung, rồi `mount()` tab mới.
- Khi đổi theme, ngôn ngữ hoặc màu: `rerender()` dựng lại toàn bộ. Cách này đơn giản và không bị sót.

> **Bài học thiết kế:** mẫu "mount trả về destroy" giúp không rò rỉ bộ nhớ hay kết nối. Mỗi thứ mở ra đều có

> chỗ đóng lại.

5.3 Lưu trạng thái người dùng (localStorage)

Mọi lựa chọn được nhớ trong trình duyệt với tiền tố `llc.`:

Khóa	Ý nghĩa
`llc.theme`	`dark` / `light`
`llc.lang`	`en` / `vi`
`llc.colors`	Màu tăng/giảm/đường/nền tùy chỉnh
`llc.tab`	Tab mở gần nhất
`llc.cot.*`	Sản phẩm, loại báo cáo, chế độ xem, khoảng thời gian, chu kỳ COT
`llc.sea.*`	Chế độ, các năm đã chọn, đường trung bình, danh sách so sánh…
`llc.charts.layout`, `llc.charts.cells`	Bố cục lưới và cấu hình từng ô chart (sản phẩm, khung, kiểu, VP, nguồn)

`store.get/set` trong `util.js` bọc `try/catch`, nên trang vẫn chạy ở chế độ ẩn danh khi localStorage bị chặn.

5.4 Theme và màu

- Màu định nghĩa bằng **biến CSS** trên `:root` (ví dụ `--amber`, `--up`, `--dn`), với hai bộ cho `data-theme="dark"` và `"light"`.
- `makeChart()` trong `util.js` đọc biến CSS khi tạo chart, nên chart tự theo theme.
- Màu người dùng chọn trong ⚙ (lưu ở `llc.colors`) được ưu tiên hơn màu mặc định.

5.5 Song ngữ

- `i18n.js` chứa hai từ điển `en` và `vi`. Hàm `t('key')` trả về chữ theo ngôn ngữ hiện tại.
- Phần tử HTML có `data-i18n="key"` được tự điền khi đổi ngôn ngữ.
- Tên sản phẩm có hai trường `en` và `vi` trong `config.js`. Hàm `name(sym)` chọn trường phù hợp.

---

6. Logic tab COT

6.1 Vị thế ròng (Net)

```
Net = Long − Short        (cho từng nhóm)
```

Ví dụ với vàng: nhóm đầu cơ Long 245.300, Short 37.421, vậy Net = **+207.879** hợp đồng (đầu cơ đang mua ròng).

6.2 Thay đổi theo tuần

```
Δ W/W = Net(tuần này) − Net(tuần trước)
```

6.3 % trên hợp đồng mở (% OI)

```
%OI Long = Long / Open Interest × 100
```

Cho biết nhóm đó nắm bao nhiêu phần trăm thị trường.

6.4 Chỉ số COT (COT Index), công thức quan trọng nhất

```
COT Index = (Net hiện tại − Net thấp nhất trong N tuần)
            ──────────────────────────────────────────── × 100
            (Net cao nhất trong N tuần − Net thấp nhất trong N tuần)
```

- N = 26, 52 hoặc 156 tuần (chọn ở mục "Chu kỳ").
- **0** nghĩa là vị thế ròng thấp nhất trong N tuần, **100** là cao nhất.
- **> 80**: nhóm đó mua ròng ở mức cực đoan (màu xanh). **< 20**: bán ròng cực đoan (màu đỏ).
- Code nằm ở hàm `indexAt(arr, i, n)` trong `cot.js`. Hàm `rollingIndex()` tính cho toàn bộ lịch sử để vẽ đường.

6.5 Biểu đồ 3 tầng (panes)

Tầng	Nội dung
0	Giá (đường), lấy từ `cotPrice`, hoặc `price` nếu không có
1	Tùy chế độ: **RÒNG** (histogram cho đầu cơ, đường cho các nhóm khác) / **MUA–BÁN** (long dương, short âm, net là đường) / **CHỈ SỐ COT** (0–100, có vạch 20 và 80)
2	Open Interest (vùng tô)

Thư viện v5 tạo nhiều tầng bằng tham số thứ ba của `chart.addSeries(Loại, tùy chọn, chỉSốTầng)`, rồi đặt tỉ lệ chiều cao bằng `pane.setStretchFactor()`.

6.6 Bảng tổng quan tất cả thị trường

- Tải song song 27 file Legacy (`Promise.all`).
- Mỗi dòng: OI, Net đầu cơ, Δ tuần, Net thương mại, Net nhỏ lẻ, chỉ số COT 52 tuần và 3 năm của đầu cơ.
- Bấm vào một dòng để mở chi tiết thị trường đó.

6.7 Đọc COT thế nào (gợi ý phân tích)

- Đầu cơ (Managed Money / Non-Commercial) thường **đi theo xu hướng**. Thương mại (Commercial) thường **đi ngược giá** vì họ phòng hộ.
- Khi chỉ số COT của đầu cơ > 80 trong khi giá đã tăng lâu, thị trường có thể "đông người mua", và rủi ro đảo chiều tăng lên.

- Nên xem kèm Open Interest: giá tăng và OI tăng là tiền mới vào. Giá tăng mà OI giảm là short đang đóng vị thế.

---

7. Logic tab Mùa vụ (Seasonal)

7.1 Chia dữ liệu theo năm: `byYear(px)`

Với mỗi ngày giao dịch trong năm Y:

```
pct = (Giá đóng cửa hôm đó / Giá đóng cửa ngày giao dịch CUỐI CÙNG của năm Y−1 − 1) × 100
```

- Dùng **giá cuối năm trước** làm mốc, giống cách tính YTD (year-to-date) chuẩn.
- Năm đầu tiên trong dữ liệu bị bỏ, vì không có năm trước để làm mốc.

7.2 Vẽ mọi năm trên cùng một lịch

Mọi ngày đều được quy về **năm 2000** (một năm nhuận, có ngày 29/2):

```
calTime("2024-03-15") = thời điểm 15/03/2000
```

Nhờ vậy đường năm 2025 và năm 2026 nằm chồng lên nhau theo ngày-tháng. Trục thời gian chỉ hiện tên tháng (`tickMarkFormatter`).

7.3 Đường trung bình N năm: `averagePath(years, n)`

1. Lấy N năm **đã kết thúc** gần nhất.
2. Với mỗi ngày trong 366 ngày lịch, lấy giá trị % của từng năm (nếu ngày đó nghỉ thì dùng giá trị gần nhất trước đó, gọi là forward-fill).
3. Lấy trung bình cộng.

7.4 Bảng lợi nhuận theo tháng / quý / năm: `periodEnds(px)`

```
Lợi nhuận tháng M = Giá cuối tháng M / Giá cuối tháng M−1 − 1
```

Quý và năm tính tương tự. Dòng **WIN %** là tỉ lệ số năm mà tháng đó tăng giá (15 năm gần nhất).

7.5 So sánh nhiều sản phẩm

- Chọn **mốc**: YTD, 1M, 3M, 6M, 1Y, 3Y, 5Y hoặc ngày tùy chọn.
- Mỗi sản phẩm: `% = Giá / Giá đóng cửa ngay trước mốc − 1` . Nhờ vậy các sản phẩm có đơn vị khác nhau (USD/oz, cent/bushel…) so sánh được trên cùng một thang %.
- **Bảng hiệu suất** sắp xếp từ tăng nhiều nhất đến giảm nhiều nhất.
- **Ma trận tương quan** dùng lợi nhuận log hằng ngày `ln(Pt / Pt−1)` , chỉ lấy những ngày cả hai sản phẩm cùng giao dịch, rồi tính hệ số Pearson (hàm `corr()` trong `util.js`).

- +1: đi cùng chiều hoàn toàn. 0: không liên quan. −1: ngược chiều hoàn toàn.

7.6 Lưới tổng quan

Mỗi sản phẩm một chart nhỏ gồm: năm nay (đậm), năm trước (xám), trung bình 5 năm (nét đứt xanh).

---

8. Logic tab Biểu đồ (Charts)

8.1 Lưới và ô chart

- Bố cục 1/2/4/6/9 ô, dùng CSS Grid (`LAYOUTS` trong `charts.js`).
- Mỗi ô do `createCell(conf)` tạo ra, độc lập hoàn toàn: có chart, nguồn dữ liệu, timer và nút phóng to riêng.
- **Phóng to (⤢):** thêm class `maxed` cho lưới và `max` cho ô. CSS ẩn các ô khác. Nhấn Esc hoặc nháy đúp tiêu đề để trở lại.
- **Toàn màn hình (⛶):** dùng Fullscreen API của trình duyệt (`element.requestFullscreen()`).

8.2 Chọn nguồn dữ liệu cho từng ô

Nguồn	Khi nào	Nhãn
AUTO (mặc định)	Sản phẩm có trên sàn: thử Binance, OKX, Bybit, Hyperliquid; nếu tất cả lỗi thì dùng Yahoo	`REALTIME · <SÀN>`
Một sàn cụ thể	Bạn tự chọn trong ô thả xuống	`REALTIME · <SÀN>`
YAHOO FUT	Giá hợp đồng tương lai thật (COMEX/NYMEX), trễ khoảng 15 phút	`TRỄ xm`
(chỉ Yahoo)	Forex, nông sản, chỉ số USD	`TRỄ`

> **Lưu ý về giá:** giá trên sàn crypto là hợp đồng **perpetual** (vĩnh cửu), bám theo giá giao ngay, nên có thể lệch vài đô so với hợp đồng tương lai COMEX/NYMEX do chênh lệch kỳ hạn (contango/backwardation). Rê chuột lên nhãn REALTIME để xem tên hợp đồng.

8.3 Ghép nến theo khung thời gian: `aggregate(bars, sec)`

Dữ liệu Yahoo có sẵn 5m và 1h. Các khung khác được ghép lại:

```
khóa nhóm = floor(time / sec) × sec              (ví dụ 15m: sec = 900)
Open  = open của nến đầu nhóm
High  = max(high), Low = min(low)
Close = close của nến cuối nhóm
Vol   = tổng volume
```

- 15m, 30m ghép từ 5m. 4H ghép từ 1h. 1W ghép từ ngày (tuần bắt đầu thứ Hai).
- 1D lấy file giá ngày, rồi **nối thêm phiên hôm nay** ghép từ nến 1h (phiên futures mới bắt đầu lúc 18:00 giờ New York).

8.4 Giờ hiển thị

Thư viện chart hiển thị giờ UTC. Để hiện giờ Việt Nam, mọi nến trong ngày được **cộng thêm độ lệch múi giờ** của máy người xem (`TZ_SHIFT = -getTimezoneOffset() × 60`).

8.5 Đồng hồ đếm ngược: `nextClose(tf, live)`

- Khung trong ngày: `ceil(now / sec) × sec` , tức mốc chia hết kế tiếp.
- 1D với sàn crypto: 00:00 UTC kế tiếp. 1D với futures: 17:00 giờ New York kế tiếp (thứ Hai đến thứ Sáu).
- 1W: crypto là thứ Hai 00:00 UTC, futures là thứ Sáu 17:00 New York.
- Nhãn đếm ngược được đặt **ngay dưới nhãn giá** bằng `series.priceToCoordinate(giá cuối)` và cập nhật mỗi giây.

8.6 Volume Profile, thuật toán chi tiết

1. Lấy các nến **đang hiển thị** trên màn hình (`getVisibleLogicalRange`).
2. Tìm giá thấp nhất `lo` và cao nhất `hi` , rồi chia thành **48 hàng giá** đều nhau.
3. Với mỗi nến, **chia đều volume của nến** cho các hàng giá nằm trong khoảng [Low, High] của nến đó.
    - Forex không có volume, nên mỗi nến tính là 1 (TPO, đếm thời gian).
4. **POC (Point of Control)** = hàng có volume lớn nhất, vẽ màu cam đậm kèm đường nét đứt.
5. **Value Area 70%**: bắt đầu từ POC, mỗi bước mở rộng sang hàng lân cận (trên hoặc dưới) có volume lớn hơn, cho tới khi gom đủ 70% tổng volume. Vùng này vẽ đậm hơn.
6. Vẽ lên một `<canvas>` phủ trên chart (`pointer-events: none` để chuột vẫn tương tác với chart), vẽ lại khi cuộn/zoom và mỗi giây.

8.7 Cập nhật dữ liệu

- **Nguồn sàn:** WebSocket đẩy nến mới liên tục và `pushBar()` cập nhật nến cuối hoặc thêm nến mới.
- **Nguồn Yahoo:** cứ 2 phút tải lại file intraday (thêm `?t=thời_gian` để bỏ qua cache), chỉ nối các nến mới.

---

9. Dữ liệu realtime đa sàn và cơ chế tự chuyển sàn

9.1 Danh mục thị trường (`MARKETS` trong `exchanges.js`)

Sản phẩm	Binance	OKX	Bybit	Hyperliquid
BTC	BTCUSDT (spot)	BTC-USDT	BTCUSDT (spot)	BTC
ETH	ETHUSDT (spot)	ETH-USDT	ETHUSDT (spot)	ETH
Vàng	XAUUSDT	XAU-USDT-SWAP	XAUUSDT	xyz:GOLD
Bạc	XAGUSDT	XAG-USDT-SWAP	XAGUSDT	xyz:SILVER
Bạch kim	XPTUSDT	XPT-USDT-SWAP	—	xyz:PLATINUM
Paladi	XPDUSDT	XPD-USDT-SWAP	—	xyz:PALLADIUM
Đồng	COPPERUSDT	XCU-USDT-SWAP	—	xyz:COPPER
Dầu WTI	CLUSDT	CL-USDT-SWAP	CLUSDT	xyz:CL
Dầu Brent	BZUSDT	BZ-USDT-SWAP	BZUSDT	xyz:BRENTOIL
Khí tự nhiên	NATGASUSDT	NG-USDT-SWAP	—	xyz:NATGAS

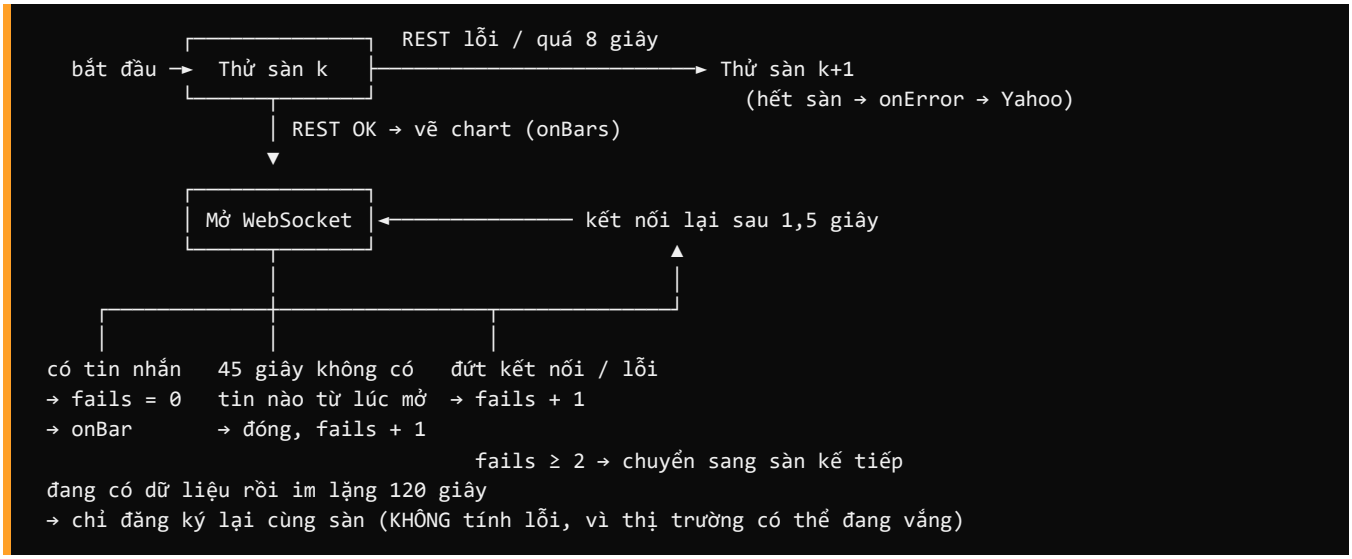
(Kim loại và năng lượng trên Binance thuộc mục TradFi của USDⓈ-M Futures. Trên Hyperliquid chúng nằm ở sàn HIP-3 tên "xyz".)

9.2 Adapter: mỗi sàn một "phiên dịch viên"

Mỗi sàn có cách gọi API riêng. `ADAPTERS` chuẩn hóa tất cả về cùng một dạng nến `{time, open, high, low, close, volume}`:

Sàn	Lấy nến cũ (REST)	Nhận nến mới (WebSocket)	Ghi chú
Binance	`/api/v3/klines` (spot), `/fapi/v1/klines` (futures)	`stream.binance.com/ws/…` và `fstream.binance.com/market/ws/…`	Futures dùng đường `/market/ws` (đường `/ws` cũ kết nối được nhưng không gửi dữ liệu)
OKX	`/api/v5/market/candles` + `history-candles` (300 nến/lần, gọi 3 trang)	`ws.okx.com:8443/ws/v5/business`, kênh `candle1H` …	Khung ngày dùng `1Dutc` để chốt nến theo UTC; ping bằng chuỗi `ping`
Bybit	`/v5/market/kline`	`stream.bybit.com/v5/public/{spot,linear}`, `kline.60.XAUUSDT`	Ping `{"op":"ping"}` mỗi 20 giây
Hyperliquid	`POST /info` `{"type":"candleSnapshot"}`	`api.hyperliquid.xyz/ws`, `{"type":"candle"}`	Ping `{"method":"ping"}`

9.3 Máy trạng thái tự chuyển sàn: `liveFeed()`



Các quy tắc then chốt:

1. **REST lỗi hoặc chậm quá 8 giây** thì chuyển ngay sang sàn tiếp theo.
2. **Mở được kết nối nhưng 45 giây không nhận được gì** thì coi là nghẽn và tính 1 lỗi. Đây chính là kiểu Binance "kết nối nhưng không gửi".
3. **2 lỗi liên tiếp** trên cùng một sàn thì chuyển sàn.
4. **Im lặng sau khi đã có dữ liệu** (thị trường vắng) thì chỉ kết nối lại cùng sàn, không chuyển.
5. Khi kết nối lại **cùng sàn**, chart chỉ được nối thêm nến mới, nên không bị mất vị trí bạn đang zoom hoặc cuộn.
6. Khi **tất cả sàn đều lỗi**: kim loại/năng lượng chuyển sang dữ liệu Yahoo (trễ), crypto hiện thông báo và thử lại sau 30 giây.

Đã kiểm thử:

- Chặn REST Binance thì tự chạy trên OKX ngay lập tức.
- Binance kết nối mà không gửi dữ liệu thì thử lại Binance ở giây 49, chuyển sang OKX ở giây 98, và nhận realtime bình thường.

9.4 Ticker tape

BTC và ETH trên dải chạy chữ dùng cùng `liveFeed()` với khung 1D. % thay đổi tính so với giá mở cửa ngày theo UTC.

---

10. Vận hành và xử lý sự cố

10.1 Việc hằng ngày: không cần làm gì

Việc	Tự động lúc (giờ VN)
COT	Thứ Bảy khoảng 04:00 (dự phòng 10:00)
Giá ngày	Mỗi ngày khoảng 05:30
Intraday + deploy	Khoảng 15 phút một lần

10.2 Chạy bằng tay

Vào GitHub, chọn tab **Actions**, chọn workflow (**Update COT**, **Update Prices** hoặc **Deploy Site**), rồi bấm **Run workflow**.

10.3 Bảng xử lý sự cố

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
Nhận email "Run failed" từ Update COT	CFTC chưa đăng file hoặc mã hợp đồng đổi	Xem log tìm dòng `MISSING`; đối chiếu mã trên trang CFTC
Nhận email từ Update Prices	Yahoo tạm lỗi	Thường tự hết ở lần chạy sau; có thể bấm chạy lại
Chart futures có khoảng trống	Yahoo trả thiếu phiên	Script đã tự thử lại; chờ lần deploy kế tiếp
Nhãn TRỄ lớn hơn 30 phút	Cron GitHub bị trễ	Bình thường lúc cao điểm; bấm **Run workflow** cho Deploy Site
Ô chart báo "Không có dữ liệu"	Mạng chặn sàn hoặc tất cả sàn lỗi	Đổi nguồn trong ô thả xuống, hoặc chờ (tự thử lại)
Lịch tự động không chạy	Repo không hoạt động 60 ngày	Actions, chọn workflow, bấm **Enable workflow**
Trang chưa thấy thay đổi mới	Cache trình duyệt	Nhấn Ctrl+F5

10.4 Giới hạn của nguồn miễn phí

- Futures thật (COMEX/NYMEX) chỉ có từ Yahoo, trễ khoảng 15–30 phút. Realtime thật chỉ có ở hợp đồng perpetual trên sàn crypto.
- Forex giao ngay không có volume tập trung.
- Không có nguồn miễn phí cho cà phê Robusta. CFTC không có COT cho Nhân dân tệ.
- Lịch cron của GitHub không đảm bảo đúng giờ tuyệt đối.

11. Hướng dẫn mở rộng

11.1 Thêm một sản phẩm mới (ví dụ: Đường số 11 — Sugar)

1. **COT:** tìm mã CFTC (Sugar No.11 = `080732`), rồi thêm vào `MARKETS` trong `scripts/fetch_cot.py`: `"SUGAR": ("080732", "softs"),`
2. **Giá:** thêm ticker Yahoo vào `TICKERS` trong `scripts/fetch_prices.py`: `"SUGAR": ("SB=F", "softs"),`
3. **Giao diện:** thêm một dòng vào `SYMBOLS` trong `js/config.js`: `{ id: 'SUGAR', code: 'SB', en: 'Sugar #11', vi: 'Đường', group: 'softs', cot: 'SUGAR', price: 'SUGAR' },`
4. Commit và push, rồi vào Actions chạy **Update COT** và **Update Prices** một lần.
5. (Tùy chọn) Nếu sàn crypto có hợp đồng tương ứng, thêm vào `MARKETS` trong `js/exchanges.js`.

11.2 Thêm một sàn mới

1. Viết một adapter trong `ADAPTERS` (`js/exchanges.js`) gồm:
   - `tf`: bảng đổi khung thời gian sang tên của sàn.
   - `bars(market, tf)`: gọi REST, trả mảng nến **cũ trước mới sau**, `time` tính bằng giây.
   - `ws(market, tf)`: trả `{url, sub, ping, parse}`.
2. Thêm tên sàn vào `EXCHANGE_ORDER` và `EXCHANGE_LABEL`.
3. Thêm mã thị trường vào `MARKETS`.
4. Kiểm tra sàn có cho phép trình duyệt gọi (header `Access-Control-Allow-Origin`).

11.3 Đổi lịch chạy

Sửa dòng `cron:` trong file `.github/workflows/*.yml`. Cú pháp `phút giờ ngày tháng thứ`, theo **giờ UTC**. Ví dụ `0 23 * * 1-5` là 23:00 UTC thứ Hai đến thứ Sáu, tức 06:00 sáng hôm sau giờ Việt Nam.

11.4 Thêm một ngôn ngữ

Thêm một từ điển mới vào `DICT` trong `i18n.js`, thêm tên tháng vào `MONTHS`, và thêm trường tên vào từng sản phẩm trong `config.js`.

---

12. Bài tập tự học

Cấp độ 1: Đọc hiểu

1. Mở `data/cot/legacy/GOLD.json` trên GitHub. Tìm Net của nhóm Non-Commercial ở báo cáo mới nhất (`nc_l - nc_s`) và đối chiếu với tab COT.
2. Trong tab Actions, mở một lần chạy **Update Prices** và tìm dòng log của vàng (`OK GOLD`).

3. Trên trang, bật chế độ CHỈ SỐ COT cho Bạc với chu kỳ 156 tuần. Hiện tại đầu cơ đang ở vùng cực đoan nào?

Cấp độ 2: Tính tay

4. Net đầu cơ của một thị trường trong 52 tuần dao động từ −20.000 đến +80.000, hiện tại là +60.000. Tính COT Index. (Đáp án: 80)
5. Giá cuối năm trước là 100, hôm nay là 112. Tính YTD %. (Đáp án: +12%)
6. Một nến có volume 900, Low = 10, High = 12, và mỗi hàng giá cao 0,5. Nến này góp bao nhiêu volume cho mỗi hàng? (Đáp án: trải trên 4–5 hàng, mỗi hàng khoảng 180–225)

Cấp độ 3: Thực hành sửa code

7. Đổi số hàng của volume profile từ 48 lên 80 (hằng `ROWS` trong `charts.js`) và xem thay đổi.
8. Thêm khung thời gian `2H` cho chart. Gợi ý: thêm vào `TF` và vào bảng `tf` của từng adapter; với Yahoo thì ghép từ 1h.
9. Thêm sản phẩm Sugar #11 theo mục 11.1.
10. Làm cho chế độ AUTO tự quay lại Binance sau 10 phút nếu Binance đã ổn định. Gợi ý: trong `liveFeed()`, khi `k > 0` thì định kỳ thử REST của sàn 0.

Cấp độ 4: Phân tích

11. Dùng tab So sánh với mốc 1Y cho Vàng, Bạc, Đồng và Dầu. Cặp nào có tương quan cao nhất, và vì sao?
12. Dùng bảng lợi nhuận theo tháng của Khí tự nhiên. Tháng nào có WIN % cao nhất? Điều này có liên quan đến mùa sưởi ấm không?

---

13. Phụ lục

13.1 Mã hợp đồng CFTC đang dùng

Sản phẩm	Mã	Sản phẩm	Mã
Vàng	088691	Dầu WTI	067651
Bạc	084691	Dầu Brent (NYMEX)	06765T
Bạch kim	076651	Khí tự nhiên	023651
Paladi	075651	Xăng RBOB	111659
Đồng	085692	Dầu sưởi ULSD	022651
Nhôm (COMEX MWP)	191693	Euro	099741
Thép HRC	192651	Yên Nhật	097741
Lúa mì SRW	001602	Bảng Anh	096742
Ngô	002602	Đô la Canada	090741
Đậu tương	005602	Đô la Úc	232741
Dầu đậu tương	007601	Franc Thụy Sĩ	092741
Khô đậu tương	026603	Chỉ số USD	098662
Cà phê Arabica	083731	Bitcoin (CME)	133741
		Ether (CME)	146021

13.2 Định dạng file dữ liệu

COT (`data/cot/legacy/GOLD.json`):

```
{ "symbol": "GOLD", "group": "metals", "market": "GOLD - COMMODITY EXCHANGE INC.",
  "report": "legacy",
  "date": ["2010-01-05", ...],
  "oi":   [...], "nc_l": [...], "nc_s": [...], "nc_sp": [...],
  "c_l":  [...], "c_s":  [...], "nr_l": [...], "nr_s": [...] }
```

Giá ngày (`data/prices/GOLD.json`):

```
{ "symbol": "GOLD", "ticker": "GC=F", "group": "metals",
  "t": ["2000-08-30", ...], "o": [...], "h": [...], "l": [...], "c": [...], "v": [...] }
```

Intraday (`data/intraday/GOLD_5m.json`): giống giá ngày nhưng `t` là giây Unix (UTC).

13.3 Từ điển thuật ngữ

Thuật ngữ	Nghĩa
COT	Commitments of Traders: báo cáo vị thế các nhà giao dịch của CFTC (Mỹ)
Open Interest (OI)	Tổng số hợp đồng đang mở
Net	Long trừ Short
Perpetual (perp)	Hợp đồng tương lai không kỳ hạn trên sàn crypto
REST API	Gọi một lần, nhận một lần (lấy lịch sử)
WebSocket	Kết nối mở liên tục, sàn tự đẩy dữ liệu mới (realtime)
CORS	Quy tắc cho phép trang web gọi API ở tên miền khác
Cron	Cú pháp đặt lịch chạy tự động
Workflow / Action	Kịch bản tự động trên GitHub
Artifact	Gói file được workflow tạo ra (ở đây là toàn bộ trang web)
POC	Point of Control: mức giá có volume lớn nhất
Value Area	Vùng giá chứa 70% volume quanh POC
YTD	Year-to-date: tính từ đầu năm
Contango / Backwardation	Giá kỳ hạn xa cao hơn / thấp hơn giá kỳ hạn gần

---

Tài liệu ứng với phiên bản mã nguồn ngày 22/10/2026.